

① class $\Rightarrow$ blue $\Rightarrow$

② $\rightarrow$ Student s = new Student()
Object

③ this keyword

④ Static vs non static method
Class name
Variable
From local
Variable
s.increase()

⑤ Static block
class obj

⑥ Constructor $\rightarrow$ class data member
in init block

St.enqueue(s2)
this

St.computer(s2)
1 memory
2 pushing
3 work

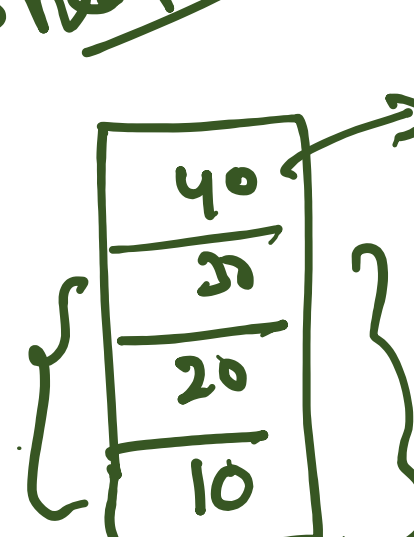
Public static() {
① memory
② pushing
③ work

① Encryption

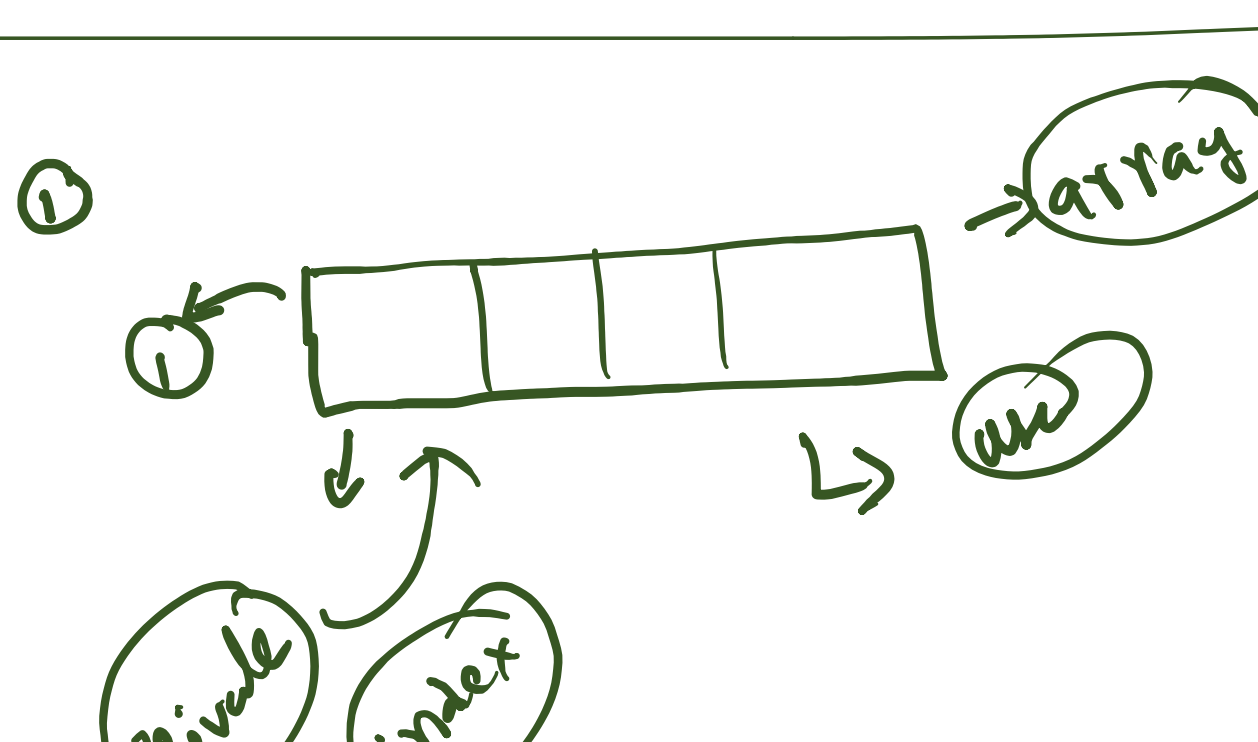
① Stack $\rightarrow$ LIFO

Class $\rightarrow$ NP $\rightarrow$ Heap

Application



add $\rightarrow$ push
remove $\rightarrow$ pop
get $\rightarrow$ peek
size $\rightarrow$ size



idx = 1
+ 1
2
3
4

idx++
arr[idx] = item
 $\Rightarrow$ arr[1+40] = 10m

St.push(10)
St.push(20)
St.push(30)
St.push(40)

public void push(int item) {
}

item = arr[idx-1] { int item = arr[idx]
idx--

as interface
as class

queue $\rightarrow$ Object

Linear queue
Circular queue

deque

sliding window

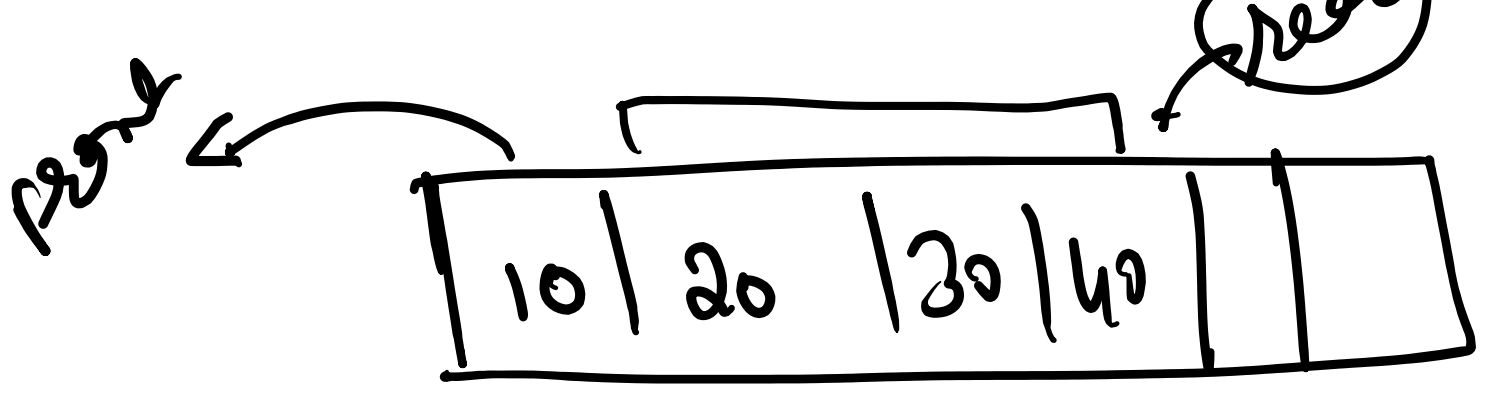
deque

FIFO

IIID

LIFO

deque



① Enqueue $\rightarrow$ Add
② Dequeue $\rightarrow$ delete
③ get front $\rightarrow$ front view
Size

public void Enqueue(int item) {
}

front = 0 size = 0
2 3 4

// 0(1)
public void Dequeue() {
}

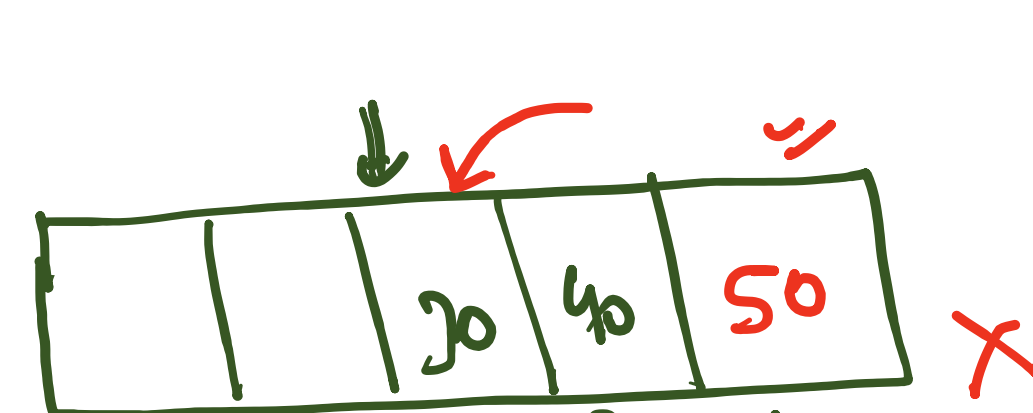


item = arr[front]
front++
size--

arr[size] = item
size++

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)

// 0(1)
public void Enqueue(int item) {
arr[size++] = item;
}
// 0(1)
public int Dequeue() {
int item = arr[front];
front++;
size--;
return item;
}



front = 0
size = 0
2 3 4

front + size
arr[front] = item
front++
size--

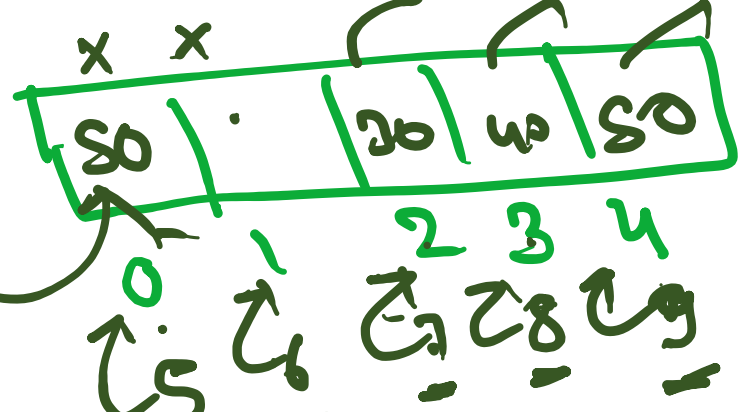
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)
q.dequeue()
q.dequeue()
q.enqueue(50)
q.enqueue(60)

Linear

idx = 0

2+2 = 4

// 0(1)
public void Enqueue(int item) {
int idx = front + size;
arr[idx] = item;
size++;
}
// 0(1)
public int Dequeue() {
int item = arr[front];
front++;
size--;
return item;
}
// 0(1)
public int getFront() {
int item = arr[front];
return item;
}



front = 0
size = 0
2 3 4

front + size
arr[front] = item
front++
size--

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)
q.dequeue()
q.dequeue()
q.enqueue(50)
q.enqueue(60)



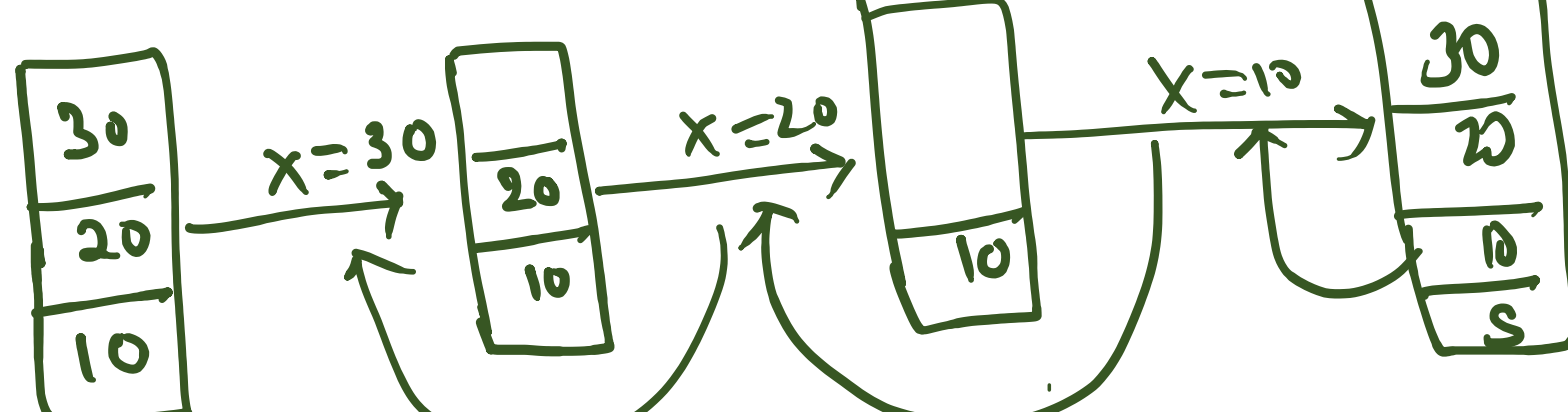
front = 2
size = 4

front = 0 i < size i++

idx = (front + 1) % arr.length

St.enqueue(x)

-5



private static void Insert(Stack<Integer> st, int item) {
// TODO Auto-generated method stub
if (st.isEmpty()) {
st.push(item);
return;
}
int x = st.pop();
Insert(st, item);
st.push(x);
}

T.C = O(n)
S.C = O(n)

